

InspectAir – Luftqualitätsmessgerät mit ESP32-S3

Echtzeit-Messung und Visualisierung von Raumluftparametern

Projektarbeit

Studiengang Elektrotechnik und Informationstechnik

Studienrichtung Fahrzeugelektronik

Duale Hochschule Baden-Württemberg Ravensburg

von

Maximilian Liebl, Cedric Stroh, Jannis Messer

Abgabedatum:	13. März 2026
Bearbeitungszeitraum:	01.11.2025 – 13.03.2026
Kurs:	Mikrocomputertechnik 1
Betreuerin / Betreuer:	Thomas Stingl (Airbus)

Erklärung

Wir versichern hiermit, dass wir die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet haben und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt haben und diese bislang nicht veröffentlicht wurde.

Dieses Dokument wurde unter Verwendung von KI-Tools (Claude, Anthropic) zur Überprüfung und Unterstützung bei der Dokumentationserstellung erstellt. Die Verwendung ist in Anhang 5.3 dokumentiert.

Ravensburg, den 13. März 2026

Maximilian Liebl

Cedric Stroh

Jannis Messer

Kurzfassung

Die vorliegende Projektarbeit beschreibt die Entwicklung des InspectAir-Systems, eines Luftqualitätsmessgeräts auf Basis des ESP32-S3-Mikrocontrollers. Das Gerät erfasst kontinuierlich fünf Luftqualitätsparameter – CO₂-Konzentration, Feinstaubbelastung (PM2.5), flüchtige organische Verbindungen (VOC), Temperatur und Luftfeuchtigkeit – und stellt diese auf einem 4-Zoll-TFT-Display mit farbcodierter Ampelbewertung dar.

Das System integriert fünf Sensoren (MH-Z19C, PMS5003, SGP40, AHT20, LD2410C) über verschiedene Buskommunikation (SPI, I²C, UART) und bietet fünf wählbare UI-Screens über das LVGL-9.2-Framework. Ein 24-GHz-Radarsensor ermöglicht die automatische Präsenzerkennung zur Steuerung des Energiemanagements (Display-Dimming und Light Sleep).

Die Arbeit folgt dem V-Modell-Entwicklungsprozess und dokumentiert den vollständigen Entwicklungszyklus von der Anforderungsspezifikation über das Hardware- und Software-Design bis zur Testspezifikation. Besondere Herausforderungen waren die Pegelproblematik bei 5V-Sensoren am 3.3V-Mikrocontroller sowie die Stabilisierung der Stromversorgung durch Stützkondensatoren.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation und Problemstellung	1
1.2	Zielsetzung	1
1.3	Aufgabenstellung	2
1.4	Team und Aufgabenverteilung	2
1.5	Aufbau der Arbeit	2
2	Grundlagen	4
2.1	ESP32-S3 Mikrocontroller	4
2.2	Sensorik	5
2.2.1	CO ₂ -Messung – MH-Z19C	5
2.2.2	Feinstaubmessung – PMS5003	5
2.2.3	VOC-Messung – SGP40	5
2.2.4	Temperatur und Luftfeuchtigkeit – AHT20	5
2.2.5	Präsenzerkennung – LD2410C	6
2.3	Display und Grafik-Framework	6
2.3.1	ST7796S TFT-Display	6
2.3.2	LovyanGFX	6
2.3.3	LVGL 9.2	6
2.4	Pegelanpassung und Schutzbeschaltung	7
2.5	Entwicklungsumgebung	7
3	Vorgehen	8
3.1	Entwicklungsprozess – V-Modell	8
3.2	Projektplanung und Zeitplan	9
3.3	Risikomanagement	10
3.4	Kostenplanung	10
3.5	Dokumentationsstruktur	10

4	Umsetzung und Ergebnisse	12
4.1	Hardware-Aufbau	12
4.1.1	Pin-Belegung	12
4.1.2	Sensor-Anbindung	12
4.1.3	Display-Hardware	14
4.1.4	Gehäusekonstruktion	14
4.2	Software-Architektur	14
4.2.1	Modularer Aufbau	14
4.2.2	Datenfluss	15
4.2.3	Sensor-Module	16
4.2.4	Sensor-Filter	16
4.3	User Interface	17
4.3.1	UI-Architektur	17
4.3.2	Farbcodierung	17
4.4	Power Management	17
4.4.1	Betriebsmodi	17
4.4.2	Backlight-Steuerung	18
4.5	Web Remote Control	18
4.6	Build-Konfiguration	19
4.7	Testergebnisse	20
4.7.1	Überblick	20
4.7.2	Lessons Learned aus den Tests	20
5	Zusammenfassung	24
5.1	Ergebnisse	24
5.2	Lessons Learned	25
5.3	Ausblick	25
	Quellenverzeichnis	27
	Abkürzungsverzeichnis	28
	Abbildungsverzeichnis	29
	Tabellenverzeichnis	30
A	Anmerkung zur Nutzung von Künstlicher Intelligenz	31

B Ergänzende Dokumentation	32
B.1 V-Modell Dokumente (linke Seite)	32
B.2 Management-Dokumente	32
B.3 V-Modell Dokumente (rechte Seite)	32
C Quellcode-Übersicht	33

1 Einleitung

1.1 Motivation und Problemstellung

Die Qualität der Raumluft hat einen erheblichen Einfluss auf Gesundheit, Wohlbefinden und Konzentrationsfähigkeit. Insbesondere in geschlossenen Räumen wie Büros, Klassenzimmern und Wohnräumen können erhöhte CO₂-Konzentrationen, Feinstaub und flüchtige organische Verbindungen zu Müdigkeit, Kopfschmerzen und langfristigen Gesundheitsschäden führen. Das Umweltbundesamt empfiehlt, die CO₂-Konzentration in Innenräumen unter 1000 ppm zu halten [9]. Gleichzeitig hat die WHO mit ihren globalen Luftqualitätsrichtlinien Grenzwerte für PM_{2.5} definiert [11].

Kommerzielle Luftqualitätsmessgeräte sind oft entweder teuer oder messen nur einzelne Parameter. Ein selbstgebautes, ESP32-basiertes System bietet die Möglichkeit, mehrere Messgrößen zu kombinieren und gleichzeitig die Prinzipien der Mikrocomputertechnik praktisch anzuwenden.

1.2 Zielsetzung

Ziel dieser Projektarbeit ist die Entwicklung eines kompakten Luftqualitätsmessgeräts, das folgende Anforderungen erfüllt:

- Kontinuierliche Messung von CO₂, Feinstaub (PM_{2.5}), VOC, Temperatur und Luftfeuchtigkeit
- Echtzeit-Darstellung aller Messwerte auf einem 4-Zoll-TFT-Display

- Farbcodierte Ampelbewertung (Grün/Gelb/Rot) nach anerkannten Grenzwerten
- Mehrere wählbare UI-Darstellungen
- Automatisches Energiemanagement mittels Radarbasierter Präsenzerkennung
- Vollständige Dokumentation gemäß V-Modell-Entwicklungsprozess

1.3 Aufgabenstellung

Die Aufgabenstellung des Moduls Mikrocomputertechnik 1 umfasst die Entwicklung eines Mikrocontroller-Projekts in Gruppenarbeit. Dabei sind die Phasen des V-Modells zu durchlaufen: Requirements Engineering, Design Description, Implementierung, Testspezifikation und Abschlussbericht. Neben dem technischen Fokus sind Kosten, Risiken, Opportunities und der Zeitplan zu berücksichtigen.

1.4 Team und Aufgabenverteilung

Das Projektteam besteht aus drei Mitgliedern:

Tabelle 1.1: Aufgabenverteilung im Team

Teammitglied	Hauptverantwortlichkeiten
Maximilian Liebl	Projektleitung, Sensor-Module, Power Management, Dokumentation
Cedric Stroh	Hardware-Integration, Level Shifter, Radar, Gehäuse (3D-Druck)
Jannis Messer	Display-Modul, LVGL-UI-Screens, LovyanGFX-Treiberanbindung

1.5 Aufbau der Arbeit

Kapitel 2 stellt die theoretischen und technischen Grundlagen des Systems vor. Kapitel 3 beschreibt den gewählten Entwicklungsprozess und die Projektplanung. In Kapitel 4 wird

die konkrete Umsetzung der Hardware- und Software-Architektur erläutert. Kapitel 5 fasst die Ergebnisse zusammen und gibt einen Ausblick auf mögliche Erweiterungen.

2 Grundlagen

2.1 ESP32-S3 Mikrocontroller

Der ESP32-S3 von Espressif Systems bildet das Herzstück des InspectAir-Systems. Es handelt sich um einen Dual-Core Xtensa LX7 Mikrocontroller mit einer Taktfrequenz von bis zu 240 MHz. Die verwendete Variante N8R8 verfügt über 8 MB Flash-Speicher und 8 MB PSRAM [2].

Wesentliche Merkmale für dieses Projekt sind:

- Mehrere UART-Schnittstellen (3 Hardware-UARTs)
- I²C und SPI Peripherie
- Integriertes WiFi und Bluetooth
- PWM-fähige GPIO-Pins
- Light-Sleep-Modus mit ca. 0,8 mA Stromaufnahme
- 3,3 V Logikpegel (kritisch für 5 V-Sensoranbindung)

Als Entwicklungsboard wird das Waveshare ESP32-S3-WROOM N8R8 DevKit eingesetzt, das USB-C-Anschluss und integrierten USB-zu-UART-Wandler bietet.

2.2 Sensorik

2.2.1 CO₂-Messung – MH-Z19C

Der MH-Z19C von Winsen ist ein NDIR-basierter CO₂-Sensor [10]. Das NDIR-Prinzip nutzt die Absorption von Infrarotstrahlung durch CO₂-Moleküle bei einer Wellenlänge von ca. 4,26 µm. Der Sensor hat einen Messbereich von 400–5000 ppm und kommuniziert über UART mit 9600 Baud. Die Versorgungsspannung beträgt 5 V, wobei beim Messvorgang Stromspitzen auftreten, die durch Stützkondensatoren (220 µF) gepuffert werden müssen.

2.2.2 Feinstaubmessung – PMS5003

Der PMS5003 von Plantower misst Feinstaubkonzentrationen mittels Laserstreuung [6]. Ein Laserstrahl wird durch die angesaugte Luft geleitet, und gestreute Photonen werden von einer Photodiode erfasst. Der Sensor liefert PM1.0-, PM2.5- und PM10-Werte in µg/m³ über UART (9600 Baud). Ein interner Lüfter sorgt für den Luftdurchsatz, was bei der Gehäusekonstruktion berücksichtigt werden muss.

2.2.3 VOC-Messung – SGP40

Der SGP40 von Sensirion ist ein Metalloxid-Gassensor für flüchtige organische Verbindungen [8]. Er kommuniziert über I²C (Adresse 0x59) und liefert einen berechneten VOC-Index (0–500). Der Sensor benötigt ca. 60 Sekunden Aufwärmzeit für stabile Messwerte und arbeitet mit 3,3 V Versorgungsspannung.

2.2.4 Temperatur und Luftfeuchtigkeit – AHT20

Der AHT20 ist ein kapazitiver Feuchte- und Temperatursensor von ASAIR [1]. Die Kommunikation erfolgt über I²C (Adresse 0x38). Der Messbereich umfasst –40 bis +85 °C (Temperatur) und 0–100 % rH (Feuchtigkeit).

2.2.5 Präsenzerkennung – LD2410C

Der LD2410C von Hi-Link ist ein 24 GHz mmWave-Radarsensor zur Präsenzerkennung [3]. Er erkennt Personen in einem Bereich von bis zu 5 m und unterscheidet zwischen Bewegung und stationärer Präsenz. Die Kommunikation erfolgt über UART mit 256000 Baud bei 5 V Versorgungsspannung. Der TX-Ausgang liefert 5 V-Signale und erfordert zwingend einen Level Shifter zum Schutz des ESP32.

2.3 Display und Grafik-Framework

2.3.1 ST7796S TFT-Display

Das 4-Zoll-TFT-Display mit ST7796S-Controller bietet eine Auflösung von 480×320 Pixeln und wird über SPI angesteuert. Die Hintergrundbeleuchtung wird über einen BS170-MOSFET per PWM gedimmt, was stufenloses Dimmen und Energiesparmodi ermöglicht.

2.3.2 LovyanGFX

LovyanGFX [4] ist eine Hardware-Abstraktionsschicht für Display-Treiber, die den direkten SPI-Zugriff mit DMA optimiert. Version 1.1.16 wird eingesetzt und bietet native Unterstützung für den ST7796S-Controller.

2.3.3 LVGL 9.2

LVGL [5] ist ein Open-Source-UI-Framework für eingebettete Systeme. Version 9.2.2 wird als Grafik-Framework genutzt und bietet Widgets wie Labels, Arcs, Bars und Container. LVGL übernimmt das Rendering und die Event-Verarbeitung der fünf UI-Screens.

2.4 Pegelanpassung und Schutzbeschaltung

Da der ESP32-S3 mit 3,3 V-Logik arbeitet, die Sensoren MH-Z19C, PMS5003 und LD2410C jedoch 5 V-Signale liefern, ist eine Pegelanpassung erforderlich. Besonders kritisch ist der LD2410C: Sein TX-Ausgang sendet mit 5 V-Pegel und würde den ESP32 ohne Level Shifter zerstören. Ein bidirektionaler Level Shifter (z.B. TXS0108E) wandelt die Signale zwischen 5 V und 3,3 V um.

Zusätzlich verursachen der MH-Z19C (während des NDIR-Messvorgangs) und der LD2410C (beim Senden) Stromspitzen, die ohne Pufferung zu Spannungseinbrüchen und Systemabstürzen führen. Elektrolytkondensatoren mit 220 μ F (25 V) an den VCC/GND-Anschlüssen dieser Sensoren stabilisieren die Versorgungsspannung.

2.5 Entwicklungsumgebung

Die Entwicklung erfolgt mit Visual Studio Code und PlatformIO [7] auf Basis des Arduino-Frameworks für die espressif32-Plattform. PlatformIO übernimmt das Dependency-Management und die Build-Konfiguration. Die Versionskontrolle erfolgt über Git mit einem privaten GitHub-Repository.

3 Vorgehen

3.1 Entwicklungsprozess – V-Modell

Für die Entwicklung des InspectAir-Systems wird das V-Modell als Vorgehensmodell eingesetzt. Dieses sequenzielle Modell eignet sich gut für Embedded-Projekte mit definierten Hardware-Anforderungen und ermöglicht eine klare Zuordnung von Entwicklungs- und Testphasen.

Die linke Seite des V-Modells umfasst die Spezifikations- und Entwurfsphasen:

1. **Requirements Specification** – Definition der funktionalen und nicht-funktionalen Anforderungen
2. **Design Description** – System- und Software-Architektur
3. **Coding Rules** – Programmierstandards und Konventionen
4. **Implementierung** – Hardware-Aufbau und Software-Entwicklung

Die rechte Seite umfasst die korrespondierenden Verifikationsphasen:

1. **Einzeltests** – Validierung einzelner Sensoren und Komponenten
2. **Integrationstests** – Zusammenspiel aller Komponenten
3. **Systemtests** – Gesamtsystem im Alltagsbetrieb

Die begleitenden Management-Dokumente (Schedule, Risk & Opportunities, Configuration Management) werden parallel geführt und regelmäßig aktualisiert.

3.2 Projektplanung und Zeitplan

Das Projekt erstreckt sich über den Zeitraum von November 2025 bis März 2026. Tabelle 3.1 zeigt die definierten Meilensteine.

Tabelle 3.1: Projektmeilensteine

MS	Datum	Meilenstein	Status
M1	Nov 2025	Pitch-Präsentation	✓
M2	Dez 2025	Hardware beschafft & Einzeltests	✓
M3	Jan 2026	Hardware-Integration komplett	🔄
M4	Feb 2026	Software fertig (alle 5 UI-Screens)	○
M5	Feb 2026	PETG-Gehäuse fertig	○
M6	Mär 2026	Testing & Dokumentation	○
M7	13.03.2026	Abgabe	Deadline

Die Projektphasen gliedern sich in:

- **Phase 1: Hardware** (Dez 2025 – Jan 2026) – Bauteile beschaffen, Einzeltests, Verkabelung
- **Phase 2: Software** (Jan – Feb 2026) – Sensor-Module, Display-Modul, LVGL-UI, Power Management
- **Phase 3: Gehäuse** (Feb 2026) – 3D-Modell in Fusion 360, PETG-Druck, Montage
- **Phase 4: Test & Dokumentation** (Mär 2026) – Integrationstests, Testprotokolle, Abschlussbericht

3.3 Risikomanagement

Tabelle 3.2 zeigt die identifizierten Risiken und die zugehörigen Maßnahmen.

Tabelle 3.2: Risikoanalyse

ID	Risiko	Stufe	Mitigation
R1	Systemabsturz durch Stromspitzen	Hoch	220 µF Stützkondensatoren
R2	ESP32-Zerstörung durch 5 V-Signale	Hoch	Level Shifter (Pflicht)
R3	Zeitplan-Verzögerung	Mittel	Puffer im Schedule
R4	Sensor-Ausfall	Mittel	Fehlertolerantes Design
R5	I ² C-Adresskonflikt	Niedrig	Verschiedene Adressen
R6	Display-Lesbarkeit	Niedrig	5 UI-Screens, große Schrift

Die kritischsten Risiken (R1, R2) wurden frühzeitig durch hardwareseitige Maßnahmen adressiert. Die Erfahrung hat gezeigt, dass insbesondere die Stützkondensatoren und der Level Shifter für den LD2410C unverzichtbar sind – ohne diese Maßnahmen kam es reproduzierbar zu Systemabstürzen bzw. Hardwareschäden.

3.4 Kostenplanung

Die Gesamtkosten des Projekts belaufen sich auf ca. 96 €. Tabelle 3.3 zeigt die Aufschlüsselung.

3.5 Dokumentationsstruktur

Die Projektdokumentation folgt dem vom Dozenten vorgegebenen Dokumentationsbaum und umfasst:

- Requirements Specification – Anforderungsspezifikation
- Design Description – System- und Softwarearchitektur

Tabelle 3.3: Kostenübersicht

Komponente	Preis ca.	Anz.
ESP32-S3 DevKit (Waveshare)	15 €	1
ST7796S 4" TFT-Display	12 €	1
MH-Z19C CO ₂ -Sensor	18 €	1
PMS5003 Feinstaubsensor	15 €	1
SGP40 VOC-Sensor	8 €	1
AHT20 Temp/Feuchte-Sensor	3 €	1
LD2410C 24 GHz Radar	5 €	1
Level Shifter, MOSFET, Passivbauteile	10 €	–
PETG-Filament für 3D-Druck-Gehäuse	10 €	1
Gesamt	96 €	

- Coding Rules – Programmierstandards
- Configuration Management – Build-Konfiguration und Versionskontrolle
- Schedule – Zeitplan und Meilensteine
- Risk & Opportunities – Risikoanalyse und Chancen
- Testspezifikation – 17 Testfälle mit Testprotokoll
- Pin-Belegung & Bauteilliste – Hardware-Dokumentation
- Projektbericht (dieses Dokument) – Gesamtbericht

4 Umsetzung und Ergebnisse

4.1 Hardware-Aufbau

4.1.1 Pin-Belegung

Die GPIO-Zuordnung des ESP32-S3 wurde sorgfältig geplant, um Konflikte mit Flash/PSRAM-Pins (GPIO 10–13 beim N8R8-Modul) und dem Boot-Pin (GPIO 0) zu vermeiden. Tabelle 4.1 zeigt die vollständige Pin-Zuordnung, die in der Header-Datei `pins.h` definiert ist.

4.1.2 Sensor-Anbindung

Die fünf Sensoren und der Radarsensor werden über drei verschiedene Bustypen angebunden:

I²C-Bus (GPIO 8/18): AHT20 (Adresse 0x38) und SGP40 (Adresse 0x59) teilen sich denselben I²C-Bus. Externe Pull-up-Widerstände (4,7 k Ω) sichern stabile Signalpegel. Da beide Sensoren unterschiedliche Adressen verwenden, treten keine Adresskonflikte auf.

UART-Sensoren: Aufgrund der begrenzten Hardware-UARTs des ESP32-S3 (UART0 für USB-Debug, UART1 für PMS5003) werden der MH-Z19C und der LD2410C über SoftwareSerial-Instanzen angebunden:

- PMS5003: Hardware UART1 (GPIO 16/17), 9600 Baud
- MH-Z19C: SoftwareSerial (GPIO 4/5), 9600 Baud

Tabelle 4.1: GPIO-Belegung des ESP32-S3

GPIO	Funktion	Beschreibung	Interface
<i>Display (SPI)</i>			
11	TFT_MOSI	Display Datenleitung	SPI
13	TFT_SCLK	Display Taktleitung	SPI
12	TFT_CS	Display Chip Select	SPI
14	TFT_MISO	Display Daten rück	SPI
9	TFT_DC	Display Data/Command	SPI
46	TFT_RST	Display Reset	SPI
3	TFT_BL	Hintergrundbeleuchtung (PWM)	PWM
<i>I²C-Bus</i>			
8	I2C_SDA	AHT20 + SGP40 Daten	I2C
18	I2C_SCL	AHT20 + SGP40 Takt	I2C
<i>UART-Sensoren</i>			
4	CO2_RX	MH-Z19C Empfang	SoftwareSerial
5	CO2_TX	MH-Z19C Senden	SoftwareSerial
16	PMS_RX	PMS5003 Empfang	UART1
17	PMS_TX	PMS5003 Senden	UART1
6	RADAR_TX	LD2410C Senden	SoftwareSerial
7	RADAR_RX	LD2410C Empfang (Level Shifter!)	SoftwareSerial
<i>Steuerung</i>			
1	UI_BUTTON	Screen-Umschalttaste	GPIO Input

- LD2410C: SoftwareSerial (GPIO 6/7), 256000 Baud

Kritische Schutzbeschaltung: GPIO 7 (RADAR_RX) empfängt 5 V-Signale vom LD2410C. Ein Level Shifter (TXS0108E) auf dieser Leitung ist **Pflicht**, da ein direkter Anschluss den ESP32 zerstören würde. Zusätzlich stabilisieren 220 µF Elektrolytkondensatoren (25 V) an den 5 V-Versorgungspins von MH-Z19C und LD2410C die Spannung gegen Einbrüche bei Stromspitzen.

4.1.3 Display-Hardware

Das ST7796S TFT-Display wird über den SPI-Bus mit bis zu 40 MHz Taktfrequenz angesteuert. Die Hintergrundbeleuchtung an GPIO 3 wird über einen BS170 N-Kanal MOSFET per PWM (44,1 kHz) stufenlos gedimmt. Dies ermöglicht sowohl die Helligkeitsregelung im Normalbetrieb als auch das Dimmen im Energiesparmodus.

4.1.4 Gehäusekonstruktion

Das Gehäuse wird als PETG-3D-Druckteil in Fusion 360 konstruiert. PETG wurde gegenüber dem ursprünglich geplanten Holzgehäuse bevorzugt, da es präzisere Passungen für Display und Sensoröffnungen ermöglicht. Das Design berücksichtigt:

- Display-Aussparung (4 Zoll) an der Frontseite
- Belüftungsöffnungen für den PMS5003-Lüfter (Pflicht für Luftdurchsatz)
- Radarsensor-Fenster (24 GHz durchlässig)
- USB-C-Zugang für Programmierung und Stromversorgung
- Montagehalterungen für das Zwei-Platinen-Design

4.2 Software-Architektur

4.2.1 Modularer Aufbau

Die Software folgt einer modularen Architektur mit klarer Trennung von Zuständigkeiten. Das ursprünglich monolithische `main.cpp` (284 Zeilen) wurde in spezialisierte Module aufgeteilt:

4.2.2 Datenfluss

Der Programmablauf in der `loop()`-Funktion folgt einem zyklischen Schema:

1. **Sensorabfrage:** Alle Sensoren werden in definierter Reihenfolge ausgelesen und die Rohwerte in die zentrale `SensorReadings`-Struktur geschrieben.
2. **Filterung:** Der `SensorFilter` glättet die Rohwerte mittels Exponential Moving Average (EMA) und stellt sicher, dass Klima-Werte (Temperatur, Feuchtigkeit) nur alle 60 Sekunden und Luftqualitätswerte nur alle 12 Sekunden aktualisiert werden.
3. **History:** Der `SensorHistory` speichert Verlaufsdaten für die grafische Darstellung.
4. **UI-Update:** Bei Änderungen wird `ui_updateSensorValues()` aufgerufen, das die geglätteten Werte an den aktiven Screen übergibt.
5. **Power Management:** Der `PowerManager` prüft die Radar-Präsenzdaten und steuert Display-Dimming bzw. Light Sleep.

Die zentrale Datenstruktur `SensorReadings` (definiert in `sensor_types.h`) bündelt alle Messwerte:

Listing 4.1: Zentrale Datenstruktur für Messwerte

```
1 struct AHTData {  
2     float temperature;  
3     float humidity;  
4 };  
5 struct SGPDData {  
6     int32_t voc_index;  
7     int32_t raw_value;  
8 };  
9 struct MHZData {  
10    int32_t co2_ppm;  
11 };  
12 struct PMSData {  
13    uint16_t PM_AE_UG_1_0;  
14    uint16_t PM_AE_UG_2_5;  
15    uint16_t PM_AE_UG_10_0;  
16 };
```

```
17 struct RadarData {
18     bool presence;
19 };
20 struct SensorReadings {
21     AHTData    aht;
22     SGPDData   sgp;
23     MHZData    mhz;
24     PMSData    pms;
25     RadarData  radar;
26 };
```

4.2.3 Sensor-Module

Jeder Sensor wird durch ein eigenes Modul gekapselt, das eine einheitliche Schnittstelle bietet: `init()`, `read()`, und Zugriffsfunktionen für die Messwerte.

MH-Z19C (CO₂): Die Kommunikation erfolgt über SoftwareSerial auf GPIO 4/5. Die MH-Z19-Library (v1.5.4) kapselt das Abfrageprotokoll. Das Modul führt automatische Baseline-Kalibrierung durch.

PMS5003 (Feinstaub): Hardware-UART1 (GPIO 16/17) wird genutzt. Die PMS Library (v1.1.0) parst die 32-Byte-Datenpakete. Der Sensor benötigt aktive Belüftung.

AHT20 + SGP40 (I²C): Beide Sensoren teilen den I²C-Bus und werden in einem kombinierten Modul `aht_sgp` zusammengefasst. Die Temperatur- und Feuchtigkeitsdaten des AHT20 werden als Kompensationswerte für den SGP40 VOC-Index verwendet.

LD2410C (Radar): SoftwareSerial auf GPIO 6/7 mit 256000 Baud. Die ld2410-Library (v0.1.3) verarbeitet die Radarframes und liefert Präsenz- und Bewegungsinformationen.

4.2.4 Sensor-Filter

Der `SensorFilter` implementiert Exponential Moving Average (EMA) zur Glättung der Sensorwerte. Unterschiedliche Update-Intervalle für verschiedene Sensortypen vermeiden unnötige Display-Updates:

- Klima-Parameter (Temperatur, Feuchtigkeit): Update alle 60 Sekunden
- Luftqualität (CO₂, VOC, PM2.5): Update alle 12 Sekunden

4.3 User Interface

4.3.1 UI-Architektur

Das UI basiert auf LVGL 9.2 mit LovyanGFX 1.1.16 als Display-Treiber. Die Verbindung zwischen beiden Bibliotheken erfolgt über einen benutzerdefinierten Flush-Callback in `lvgl_driver.cpp`, der die LVGL-Pixel Daten an das LovyanGFX-Backend übergibt.

Die UI-Architektur unterstützt fünf Screens, die über einen physischen Taster (GPIO 1) oder die Web-Fernsteuerung zyklisch durchgeschaltet werden. Jeder Screen ist als eigenständiges LVGL-Screen-Objekt implementiert:

4.3.2 Farbcodierung

Alle Screens verwenden eine einheitliche Ampel-Farbcodierung basierend auf Grenzwerten nach UBA und WHO:

Die Farbbestimmung ist in `colors.h` als Klassifizierungsfunktionen implementiert, die von allen Screens einheitlich genutzt werden.

4.4 Power Management

4.4.1 Betriebsmodi

Das Power Management nutzt den LD2410C-Radarsensor zur automatischen Erkennung von Personen im Raum und steuert drei Betriebsmodi:

1. **Aktiv** (Präsenz erkannt): Display volle Helligkeit ($255/255 = 100\%$), alle Sensoren aktiv, UI-Updates normal.
2. **Gedimmt** (keine Präsenz seit 45s): Display auf $30/255$ ($\approx 12\%$) reduziert, Sensoren weiter aktiv.
3. **Light Sleep** (keine Präsenz seit erweitertem Timeout): ESP32 in Light Sleep ($\approx 0,8\text{ mA}$), Aufwachzeit $\approx 2\text{ ms}$ bei Radar-Trigger.

Light Sleep wurde gegenüber Deep Sleep gewählt, da bei Deep Sleep der RAM verloren geht, was zum Verlust der Sensorhistorie und der Kalibrierungswerte des SGP40 führen würde. Zudem ermöglicht Light Sleep eine schnellere Reaktionszeit auf Radar-Events.

4.4.2 Backlight-Steuerung

Die Display-Hintergrundbeleuchtung wird über GPIO 3 per PWM gesteuert:

Listing 4.2: PWM-Konfiguration für Backlight

```

1 #define BL_PIN          3
2 #define PWM_CHANNEL     0
3 #define PWM_FREQUENCY   44100 // 44.1 kHz
4 #define PWM_RESOLUTION  8     // 8-bit (0-255)
5 #define BRIGHTNESS_ACTIVE 255 // 100%
6 #define BRIGHTNESS_DIMMED 30  // ~12%
7 #define DIM_TIMEOUT_MS  45000 // 45 Sekunden

```

4.5 Web Remote Control

Über die WiFi-Anbindung des ESP32-S3 stellt das System einen minimalen Webserver bereit, der eine Browser-basierte Fernsteuerung ermöglicht. Die Funktionalität umfasst:

- Screen-Umschaltung (alle 5 Screens)
- Aktuelle Messwerte als JSON-API

- Display-Helligkeit einstellen

Die Implementierung nutzt den asynchronen Webserver des ESP32 und liefert eine einfache HTML-Seite an mobile Endgeräte.

4.6 Build-Konfiguration

Die zentrale Build-Konfiguration erfolgt über `platformio.ini`:

Listing 4.3: PlatformIO-Konfiguration (Auszug)

```
1 [env:esp32s3]
2 platform = espressif32
3 board = esp32-s3-devkitc-1
4 framework = arduino
5 board_build.arduino.memory_type = qio_opi
6 board_build.partitions = huge_app.csv
7 monitor_speed = 115200
8 lib_deps =
9     lvgl/lvgl@^9.2.2
10    lovyan03/LovyanGFX@^1.1.16
11    adafruit/Adafruit AHTX0@^2.0.5
12    adafruit/Adafruit SGP40@^1.0.0
13    fu-hsi/PMS Library@^1.1.0
14    jonathan-casey/MH-Z19@^1.5.4
15    ncmreynolds/ld2410@^0.1.3
16    plerup/EspSoftwareSerial@^8.2.0
17 build_flags =
18     -D LV_CONF_SKIP
19     -D LV_COLOR_DEPTH=16
20     -D LV_TICK_PERIOD_MS=5
21     -D BOARD_HAS_PSRAM
22     -mfix-esp32-psram-cache-issue
```

4.7 Testergebnisse

4.7.1 Überblick

Die Testspezifikation umfasst 17 Testfälle in sechs Kategorien. Die Einzelsensor-Tests (TC-01 bis TC-06) wurden erfolgreich durchgeführt und bestätigen die korrekte Funktion aller Sensormodule. Tabelle 4.5 gibt eine Übersicht.

4.7.2 Lessons Learned aus den Tests

Aus den bisherigen Tests ergaben sich wichtige Erkenntnisse:

Level Shifter ist Pflicht: Beim ersten Anschluss des LD2410C ohne Level Shifter wurde der GPIO des ESP32 beschädigt. Der Level Shifter auf der RX-Leitung (GPIO 7) ist daher als kritische Schutzmaßnahme dokumentiert.

Stützkondensatoren sind Pflicht: Ohne die 220 µF-Kondensatoren an MH-Z19C und LD2410C kam es reproduzierbar zu Spannungseinbrüchen und Systemabstürzen. Die Kondensatoren müssen möglichst nah an den VCC-Pins der Sensoren platziert werden.

GPIO-Auswahl ist kritisch: GPIO 0 verursacht Boot-Probleme bei Pull-down. Der UI-Button wurde daher auf GPIO 1 verlegt. GPIOs 10–13 sind beim N8R8-Modul für Flash/PSRAM reserviert und dürfen nicht verwendet werden.

SoftwareSerial für Hochgeschwindigkeit: Der LD2410C mit 256000 Baud über SoftwareSerial war zunächst problematisch. Stabile Kommunikation wurde erst nach Optimierung der Puffer und Timing-Parameter erreicht.

Tabelle 4.2: Software-Module und Dateien

Modul	Dateien	Funktion
Zentrale	main.cpp	Setup, Loop, Sensor- abfrage, UI- Update
Konfiguration	pins.h, colors.h, sensor_types.h	Pin- Definitionen, Farben, Daten- struk- turen
Display	lvgl_driver.h/cpp, display_config.h	LovyanGFX- Treiber, LVGL- Initialisierung
UI-Screens	screen_*.h/cpp, ui_manager.h/cpp	5 UI- Screens, Screen- Management
Sensoren	aht_sgp.h/cpp, mhz19c.h/cpp, pms5003.h/cpp, ld2410c.h/cpp	Sensorabstraktion
Utilities	sensor_filter.h/cpp, sensor_history.h/cpp	Glättung, Ver- laufsda- ten
Power	power_manager.h/cpp	Dimming, Light Sleep, Watch- dog
Netzwerk	WifiClock.h/cpp, web_remote.h/cpp	NTP- Zeit, Web- Fernsteuerung

Tabelle 4.3: UI-Screens des InspectAir-Systems

Nr.	Screen	Beschreibung
1	Tree-Screen	Animierter Baum, dessen Farbe die Gesamtluftqualität widerspiegelt (Grün→Braun). Stimmungsbild-Darstellung.
2	Overview-Screen	Großer AQI-Kreis mit Temperatur- und Feuchtigkeitskacheln. Kompakte Gesamtübersicht.
3	Detail-Screen	Kleinerer AQI-Kreis mit vier Detailkacheln (CO ₂ , VOC, PM2.5, Temp/Feuchte).
4	Analog-Screen	Analoges Instrumenten-Layout mit Zeigern und Skalen für jeden Messwert.
5	Bubble-Screen	Dynamische Blasen-Darstellung mit größencodierten Messwerten.

Tabelle 4.4: Grenzwerte für die Farbcodierung

Parameter	Grün (gut)	Gelb (mäßig)	Rot (schlecht)
CO ₂	< 800 ppm	800–1200 ppm	> 1200 ppm
PM2.5	< 12 µg/m ³	12–35 µg/m ³	> 35 µg/m ³
VOC-Index	< 100	100–250	> 250
Temperatur	18–24 °C	15–18 / 24–28 °C	< 15 / > 28 °C
Feuchtigkeit	40–60 %	30–40 / 60–70 %	< 30 / > 70 %

Tabelle 4.5: Übersicht Testergebnisse

Test-ID	Beschreibung	Status
TC-01	AHT20 Temperatur & Feuchtigkeit	PASS
TC-02	SGP40 VOC-Index	PASS
TC-03	I ² C-Bus (AHT20 + SGP40 parallel)	PASS
TC-04	MH-Z19C CO ₂ -Messung	PASS
TC-05	PMS5003 Feinstaubmessung	PASS
TC-06	LD2410C Radarsensor (mit Level Shifter)	PASS
TC-07	Display & Backlight PWM	ausstehend
TC-08	Stromversorgung (Stützkondensatoren)	ausstehend
TC-09	Integration: Alle Sensoren parallel	ausstehend
TC-10	Screen-Darstellung (alle 5 Screens)	ausstehend
TC-11	Screen-spezifische Features	ausstehend
TC-12	Button-Wechsel (zyklisch)	ausstehend
TC-13	Messwerte-Konsistenz über Screens	ausstehend
TC-14	Display-Dimming bei Abwesenheit	ausstehend
TC-15	Light-Sleep und Aufwachverhalten	ausstehend
TC-16	Watchdog-Timer	ausstehend
TC-17	24h-Dauerlauf	ausstehend

5 Zusammenfassung

5.1 Ergebnisse

Im Rahmen dieser Projektarbeit wurde das InspectAir-System erfolgreich konzipiert und weitgehend umgesetzt. Das Ergebnis ist ein funktionsfähiges Luftqualitätsmessgerät auf Basis des ESP32-S3-Mikrocontrollers, das fünf Luftqualitätsparameter in Echtzeit erfasst und auf einem 4-Zoll-TFT-Display darstellt.

Die wesentlichen Projektergebnisse lassen sich wie folgt zusammenfassen:

Hardware: Alle fünf Sensoren (AHT20, SGP40, MH-Z19C, PMS5003, LD2410C) wurden erfolgreich angebunden und in den Einzeltests validiert. Die kritischen Schutzbeschaltungen (Level Shifter, Stützkondensatoren) haben sich als unverzichtbar erwiesen und wurden entsprechend dokumentiert. Die Pin-Belegung des ESP32-S3 wurde optimiert, um Konflikte mit reservierten Pins zu vermeiden.

Software: Die modulare Softwarearchitektur mit über 29 Quelldateien ermöglicht eine klare Trennung der Zuständigkeiten. Das LVGL-9.2-basierte UI bietet fünf verschiedene Screen-Darstellungen mit einheitlicher Ampel-Farbcodierung. Der SensorFilter sorgt für geglättete Messwerte, und das Power Management nutzt den Radarsensor für automatisches Energiesparen.

Dokumentation: Der gesamte Entwicklungszyklus wurde gemäß V-Modell dokumentiert: von der Requirements Specification über die Design Description und Coding Rules bis zur Testspezifikation mit 17 Testfällen. Die Dokumentationsstruktur folgt den Vorgaben der Aufgabenstellung.

5.2 Lessons Learned

Mehrere wichtige Erkenntnisse wurden im Verlauf des Projekts gewonnen:

Der **Level Shifter für den LD2410C** erwies sich als das kritischste Hardware-Thema. Der direkte Anschluss eines 5 V-UART-Signals an den 3,3 V-GPIO führte zur Beschädigung des Pins. Diese Erfahrung unterstreicht die Bedeutung einer sorgfältigen Pegelanalyse bei der Schnittstellenplanung.

Die **Stützkondensatoren** für MH-Z19C und LD2410C waren erst nach systematischem Debugging als notwendig identifiziert worden. Sporadische Abstürze ohne erkennbares Muster erwiesen sich als Spannungseinbrüche bei gleichzeitigen Stromspitzen. Die 220 μ F-Kondensatoren lösten das Problem vollständig.

Die **Wahl von Light Sleep statt Deep Sleep** war eine bewusste Designentscheidung. Deep Sleep hätte zwar weniger Strom verbraucht, aber den Verlust aller Sensorhistorie und Kalibrierungswerte bedeutet. Light Sleep bietet mit ca. 0,8 mA und 2 ms Aufwachzeit einen guten Kompromiss.

Die **GPIO-Planung beim ESP32-S3** erfordert besondere Aufmerksamkeit: Die Pins 10–13 sind beim N8R8-Modul für Flash/PSRAM reserviert, GPIO 0 verursacht Boot-Probleme, und die Gesamtzahl verfügbarer GPIOs wird durch die vielen Peripheriegeräte stark eingeschränkt.

5.3 Ausblick

Folgende Erweiterungen sind für zukünftige Versionen denkbar:

WiFi-Dashboard: Die bestehende Web-Remote-Funktion könnte zu einem vollwertigen Browser-Dashboard ausgebaut werden, das Echtzeit-Messwerte und Verlaufsgrafiken anzeigt.

Batteriebetrieb: Mit einem LiPo-Akku und optimiertem Power Management könnte das Gerät auch mobil eingesetzt werden.

SD-Karten-Logging: Langzeitaufzeichnungen auf eine SD-Karte würden nachträgliche Auswertungen und Korrelationsanalysen ermöglichen.

Alarm-Funktion: Ein integrierter Piezo-Summer könnte bei kritischen Luftqualitätswerten einen akustischen Alarm auslösen.

Custom PCB: Das aktuell auf Steckbrettern/Prototypenplatinen aufgebaute System könnte auf eine professionelle Leiterplatte überführt werden, um die Zuverlässigkeit zu erhöhen und die Baugröße zu reduzieren.

Quellenverzeichnis

- [1] ASAIR. *AHT20 – Integrated Temperature and Humidity Sensor*. 2020.
- [2] Espressif Systems. *ESP32-S3 Series Datasheet*. 2023. URL: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf.
- [3] Hi-Link Electronics. *HLK-LD2410C 24GHz mmWave Radar Sensor*. 2023.
- [4] lovyan03. *LovyanGFX – SPI LCD Graphics Library for ESP32*. 2024. URL: <https://github.com/lovyan03/LovyanGFX> (besucht am 15.01.2026).
- [5] LVGL LLC. *LVGL Documentation – Version 9.2*. 2025. URL: <https://docs.lvgl.io/9.2/> (besucht am 01.02.2026).
- [6] Plantower. *PMS5003 Digital Universal Particle Concentration Sensor*. Version 2.3. 2016.
- [7] PlatformIO Labs. *PlatformIO – Professional Collaborative Platform for Embedded Development*. 2025. URL: <https://platformio.org/> (besucht am 10.01.2026).
- [8] Sensirion AG. *SGP40 – Indoor Air Quality Sensor for VOC Measurements*. 2020.
- [9] Umweltbundesamt. *Gesundheitliche Bewertung von Kohlendioxid in der Innenraumluft*. 2008. URL: https://www.umweltbundesamt.de/sites/default/files/medien/pdfs/kohlendioxid_2008.pdf (besucht am 20.01.2026).
- [10] Winsen Electronics. *MH-Z19C CO2 Module Manual*. Version 1.0. 2020.
- [11] World Health Organization. *WHO Global Air Quality Guidelines*. 2021. URL: <https://www.who.int/publications/i/item/9789240034228> (besucht am 20.01.2026).

Abkürzungsverzeichnis

DMA Direct Memory Access 6

GPIO General Purpose Input/Output 4

I²C Inter-Integrated Circuit 4, 5

KI Künstliche Intelligenz 30

LVGL Light and Versatile Graphics Library 6

NDIR Non-Dispersive Infrared 5

PETG Polyethylenterephthalatglycol 9, 14

PSRAM Pseudo Static Random Access Memory 4

PWM Pulsweitenmodulation 4, 6

SPI Serial Peripheral Interface 4, 6

UART Universal Asynchronous Receiver/Transmitter 4–6

UI User Interface 6

Abbildungsverzeichnis

Tabellenverzeichnis

1.1	Aufgabenverteilung im Team	2
3.1	Projektmeilensteine	9
3.2	Risikoanalyse	10
3.3	Kostenübersicht	11
4.1	GPIO-Belegung des ESP32-S3	13
4.2	Software-Module und Dateien	21
4.3	UI-Screens des InspectAir-Systems	22
4.4	Grenzwerte für die Farbcodierung	22
4.5	Übersicht Testergebnisse	23
A.1	Liste der verwendeten KI-basierten Werkzeuge	31

A Anmerkung zur Nutzung von Künstlicher Intelligenz

Im Rahmen dieser Arbeit wurden KI-basierte Werkzeuge benutzt. Tabelle A.1 gibt eine Übersicht über die verwendeten Werkzeuge und den jeweiligen Einsatzzweck.

Tabelle A.1: Liste der verwendeten KI-basierten Werkzeuge

Werkzeug	Beschreibung der Nutzung
Claude (Anthropic)	• Unterstützung beim Code-Refactoring (Aufteilung monolithisches main.cpp in Module) • Generierung von LaTeX-Dokumenten aus DOCX-Vorlagen • Überprüfung und Korrektur der Pin-Belegungstabellen gegen den Quellcode • Hilfe bei der Erstellung der Testspezifikation
GitHub Copilot	• Code-Completion beim Schreiben der Sensor-Module • Unterstützung bei Boilerplate-Code für LVGL-Widgets
DeepL	• Übersetzung englischer Datenblätter und Bibliotheksdokumentationen

Alle KI-generierten Inhalte wurden von den Autoren überprüft, validiert und bei Bedarf angepasst. Die technischen Entscheidungen und die Verantwortung für die Korrektheit liegen bei den Autoren.

B Ergänzende Dokumentation

Die folgenden Dokumente sind als separate Dateien im Engineering Folder enthalten und ergänzen diesen Bericht:

B.1 V-Modell Dokumente (linke Seite)

1. **Requirements Specification** (v2.0) – Funktionale und nicht-funktionale Anforderungen mit Akzeptanzkriterien
2. **Design Description** (v2.0) – System- und Softwarearchitektur, Hardware-Blockdiagramm
3. **Coding Rules** (v2.0) – Programmierstandards, Namenskonventionen, Git-Workflow

B.2 Management-Dokumente

1. **Configuration Management** (v2.0) – Build-Konfiguration, Library-Versionen, Pin-Konfiguration
2. **Schedule** (v2.0) – Projektplan mit 7 Meilensteinen, Phasen und Gantt-Übersicht
3. **Risk & Opportunities** (v2.0) – 6 Risiken, 5 Opportunities, Kostenübersicht

B.3 V-Modell Dokumente (rechte Seite)

1. **Testspezifikation** (v2.0) – 17 Testfälle in 6 Kategorien (Sensor, Hardware, Integration, UI, Power, System)
2. **Pin-Belegung & Bauteilliste** (v2.0) – Vollständige GPIO-Zuordnung und Bill of Materials

C Quellcode-Übersicht

Das InspectAir-Projekt umfasst 29 Quelldateien. Die vollständige Codebase befindet sich im Git-Repository. Nachfolgend eine Übersicht der Verzeichnisstruktur:

```
1 include/
2     pins.h                Pin-Definitionen (alle GPIO)
3     colors.h              Farben & Klassifizierungsfunktionen
4     display_config.h      LovyanGFX LGFX-Klasse
5     sensor_types.h        Datenstrukturen
6
7 src/
8     main.cpp              Zentrale Loop (~110 Zeilen)
9     display/
10         lvgl_driver.h/cpp  LVGL-Initialisierung
11         ui_manager.h/cpp   Screen-Management
12         screen_tree.h/cpp  Screen 1: Baum-Animation
13         screen_overview.h/cpp Screen 2: AQI-Uebersicht
14         screen_detail.h/cpp Screen 3: Detail-Kacheln
15         screen_analog.h/cpp Screen 4: Analog-Instrumente
16         screen_bubble.h/cpp Screen 5: Blasen-Darstellung
17     sensors/
18         aht_sgp.h/cpp      AHT20 + SGP40 (I2C)
19         mh_z19c.h/cpp     MH-Z19C CO2 (SoftwareSerial)
20         pms5003.h/cpp     PMS5003 Feinstaub (UART1)
21         ld2410c.h/cpp     LD2410C Radar (SoftwareSerial)
22     utils/
23         sensor_filter.h/cpp EMA-Glaettung
24         sensor_history.h/cpp Verlaufsdaten
25         power_manager.h/cpp Dimming & Sleep
26     WifiClock.h/cpp       NTP-Zeitsynchronisation
27     web_remote.h/cpp       Browser-Fernsteuerung
```